Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2
Бирюков С. С.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 7
3.1. Описание структуры программы 7
3.2. Описание структур данных 7
3.3. Описание алгоритмов 8
4. Результаты экспериментов и тестирования 11
Заключение 12
Литература 13

Введение

При проектировании программы бывает так, что возникает необходимость в использовании флагов, характеризующих определенные данные. Иногда при большом объеме этих данных складывается целый массив формата да/нет, и, чтобы эффективно его использовать, нужно что-то более оптимизированное по сравнению обычных bool переменных, которые занимают по 1 байту памяти. Для решения подобной задачи мы создаем класс битовой строки, с помощью которой можно хранить массив флагов, где каждый элемент будет занимать всего 1 бит. И для более удобного использования создаем класс множества, которое должно характеризоваться битовым полем. Кроме того в процессе выполнения лабораторной работы мы осваиваем написание Google-тестов для проверки корректности работы программы

1. Постановка задачи

Подробнее описание постановки задачи.

Необходимо реализовать на языке программирования C++ два класса по данному шаблону методов:

- TBitField – класс, содержащий в себе массив переменных типа unsigned int, которые в машинном представлении являются последовательностью битов, которые как раз таки и будут отражать в себе нужные данные. В интерфейс входит установка, сброс и чтение отдельных битов по их индексу, а также основные побитовые операции;
- TSet – класс, представляющий собой математическое множество, внутри себя содержит объект TBitField. Элемент i принадлежит множеству тогда и только тогда, когда i-й бит в классе равен 1. Класс должен реализовывать теоретико-множественные операции: объединение, пересечение, разность, дополнение.
- Дополнительно требуется написать Unit-тесты с использованием библиотеки Google Test для проверки корректности работы всех методов, включая краевые случаи

2. Руководство пользователя

Класс TBitField (Битовое поле)

Конструкторы и деструктор:

- TBitField(int len) – создает битовое поле заданной длины len. Выделяет память и инициализирует все биты нулями. Если len < 0, выбрасывает исключение.
- TBitField(const TBitField& bf) – конструктор копирования. Создает копию существующего поля bf.
- ~TBitField() – деструктор. Освобождает выделенную динамическую память.

Доступ к битам:

- int GetLength(void) const – возвращает логическую длину поля (количество битов).
- void SetBit(const int n) – устанавливает бит с индексом n в значение 1. Если индекс n выходит за пределы, выбрасывает исключение.
- void ClrBit(const int n) – сбрасывает бит с индексом n в значение 0. Если индекс n выходит за пределы, выбрасывает исключение.
- int GetBit(const int n) const – возвращает значение бита с индексом n (1 или 0). При выходе за границы выбрасывает исключение.

Перегруженные операции:

- int operator==(const TBitField& bf) const – возвращает 1, если поля равны (совпадают по длине и содержимому), иначе 0.
- int operator!=(const TBitField& bf) const – возвращает 1, если поля различаются, иначе 0.
- TBitField& operator=(const TBitField& bf) – оператор присваивания. Копирует данные из bf в текущий объект. Возвращает ссылку на текущий объект для поддержки цепочек присваивания.
- TBitField operator|(const TBitField& bf) – возвращает новое битовое поле, являющееся результатом побитового ИЛИ (объединение).
- TBitField operator&(const TBitField& bf) – возвращает новое битовое поле, являющееся результатом побитового И (пересечение).
- TBitField operator~(void) – возвращает новое битовое поле, в котором все биты инвертированы (логическое НЕ).
- std::ostream& operator<<(std::ostream& ostr, const TBitField& bf) – перегруженный оператор вывода. Выводит битовую строку (последовательность нулей и единиц) в поток.
- std::istream& operator>>(std::istream& istr, TBitField& bf) – перегруженный оператор ввода. Считывает строку из нулей и единиц, перестраивает поле под новую длину и заполняет его.

2.2. Класс TSet (Множество)

Конструкторы и преобразования:

- TSet(int mp) – создает пустое множество с мощностью mp. Если mp < 0, выбрасывает исключение.
- TSet(const TSet& s) – конструктор копирования.
- TSet(const TBitField& bf) – конструктор преобразования. Создает множество на основе готового битового поля.
- operator TBitField() – оператор преобразования типа. Позволяет неявно получить копию внутреннего битового поля из объекта множества.

Доступ к элементам:

- int GetMaxPower(void) const – возвращает максимальную мощность множества.
- void InsElem(const int Elem) – включает элемент Elem в множество. Если элемент вне универсума, выбрасывает исключение.

- void DelElem(const int Elem) – удаляет элемент Elem из множества. Если элемент вне универсума, выбрасывает исключение.
- int IsMember(const int Elem) const – проверяет наличие элемента в множестве. Возвращает 1, если элемент есть, и 0, если нет.

Перегруженные операции:

- int operator==(const TSet& s) const – возвращает 1, если множества равны (совпадают их универсумы и элементы).
- int operator!=(const TSet& s) const – возвращает 1, если множества не равны.
- TSet& operator=(const TSet& s) – оператор присваивания.
- TSet operator+(const int Elem) – возвращает новое множество, которое является копией текущего, но с добавленным элементом Elem.
- TSet operator-(const int Elem) – возвращает новое множество, которое является копией текущего, но без элемента Elem.
- TSet operator+(const TSet& s) – возвращает объединение двух множеств.
- TSet operator*(const TSet& s) – возвращает пересечение двух множеств.
- TSet operator~(void) – возвращает дополнение множества (все элементы универсума, которых нет в текущем множестве).
- std::ostream& operator<<(std::ostream& ostr, const TSet& s) – вывод множества в формате {1, 3, 5}.
- std::istream& operator>>(std::istream& istr, TSet& s) – ввод множества из строки.

3. Руководство программиста

3.1. Описание структуры программы

Описание должно содержать то, из каких модулей состоит программа, назначение модулей, состав модулей, какие функции и классы содержатся в них.

3.2. Описание структур данных

В программе используются два пользовательских класса (TBitField и TSet) и один базовый псевдоним типа (TELEM). Все поля классов являются приватными. Глобальные переменные отсутствуют.

Базовый тип TELEM

typedef unsigned int TELEM – псевдоним для беззнакового целого числа. Используется как базовое машинное слово для хранения битов. В текущей конфигурации занимает 32 бита (4 байта). Выбор данного типа позволяет упаковывать 32 независимых логических значения (флага) в одну переменную, что обеспечивает компактность хранения.

Класс TBitField (Битовое поле)

Класс предназначен для низкоуровневого управления набором битов.

Поля класса:

- int BitLen – логическая длина поля. Определяет максимальное количество битов, доступных пользователю. Используется для проверки корректности вводимых индексов (защита от выхода за границы).
- int MemLen – количество элементов в массиве памяти. Вычисляется по формуле (BitLen + 31) / 32 (округление вверх). Необходимо для выделения корректного объема памяти и организации циклов обработки массива.
- TELEM *pMem – указатель на динамический массив машинных слов. Хранит физическое представление битового поля. Память выделяется в конструкторе (new) и освобождается в деструкторе (delete[]).

Вспомогательные методы:

- int GetMemIndex(int n) const – вычисляет индекс элемента в массиве pMem, в котором находится бит с номером n (операция n / 32).
- TELEM GetMemMask(int n) const – генерирует битовую маску, содержащую единицу в позиции запрашиваемого бита внутри машинного слова (операция 1 << (n % 32)).

Класс TSet (Множество)

Класс реализует структуру данных «множество» на основе битового поля.

Поля класса:

- int MaxPower – максимальная мощность множества (размер универсума). Задает диапазон допустимых элементов (от 0 до MaxPower - 1). Используется для валидации входных данных в методах добавления, удаления и проверки элементов.
- TBitField BitField – характеристический вектор множества. Представляет собой объект класса TBitField. Хранит информацию о принадлежности элементов: если бит с индексом i равен 1, элемент i присутствует в множестве.

Композиция:

Класс TSet использует композицию: он содержит объект TBitField в качестве поля. При создании объекта TSet(mp) в списке инициализации конструктора автоматически вызывается конструктор TBitField(mp). Это обеспечивает выделение необходимой памяти и инициализацию битов нулями (создание пустого множества) еще до входа в тело конструктора TSet.

3.3. Описание алгоритмов

Класс TBitField

Конструктор TBitField(int len)

Алгоритм: проверяет длину на отрицательное значение. Вычисляет физический размер массива по формуле MemLen = (len + 31) / 32 (округление вверх при целочисленном делении). Выделяет память под массив с помощью оператора new и в цикле инициализирует все его элементы нулями.

Сложность: O(N), так как требуется пройтись циклом по всем выделенным словам для их обнуления.

Конструктор копирования и оператор присваивания (operator=)

Сложность: O(N) из-за необходимости скопировать все слова массива в новую память.

Формулы доступа к битам (Вспомогательные методы)

Алгоритм: чтобы найти нужный бит, используются две формулы.

- Номер элемента в массиве: n / 32 (так как в одном элементе 32 бита).
- Маска (сдвиг): 1 << (n % 32). Остаток от деления n % 32 дает позицию бита внутри числа (от 0 до 31).

Сложность: O(1), операции выполняются мгновенно.

Установка, сброс и чтение бита (SetBit, ClrBit, GetBit)

Алгоритм: сначала проверяется, что индекс n не выходит за пределы [0, BitLen-1]. Затем вычисляется индекс элемента и маска.

- SetBit: применяет побитовое ИЛИ (pMem[idx] |= mask), которое принудительно ставит 1 в нужную позицию.
- ClrBit: применяет побитовое И с инвертированной маской (pMem[idx] &= ~mask), сбрасывая бит в 0.
- GetBit: применяет побитовое И (pMem[idx] & mask). Если результат не равен нулю, возвращает 1, иначе 0.

Сложность: O(1).

Побитовые операции | (ИЛИ) и & (И)

Алгоритм: сначала определяется длина результирующего поля как максимум из длин двух операндов (чтобы вместить все данные). Создается новое поле. В цикле по всем словам выполняется соответствующая побитовая операция над парами элементов из первого и второго поля. Если одно поле короче, его недостающие слова считаются нулями.

Сложность: O(N), где N – количество слов в самом длинном операнде.

Оператор отрицания ~ (НЕ)

Алгоритм: создается поле той же длины. В цикле каждое слово инвертируется побитово (~pMem[i]).

Особенность: если длина поля не кратна 32, в последнем слове после инверсии образуются «мусорные» единицы в неиспользуемых битах. Чтобы их убрать, вычисляется остаток tail = BitLen % 32. Создается маска из tail младших единиц (1 << tail) - 1, и последнее слово логически умножается на нее (& mask).

Сложность: O(N).

Ввод и вывод (<<, >>)

Алгоритм: при выводе в цикле от 0 до BitLen вызывается GetBit и результат печатается. При вводе считывается строка, проверяется ее длина (если не совпадает с текущей, память пересоздается), затем символы '1' и '0' преобразуются в вызовы SetBit.

Сложность: O(BitLen) – зависит от количества выводимых/вводимых битов.

Класс TSet

Конструкторы

Алгоритм: В конструкторе TSet(int mp) используется список инициализации : BitField(mp), который сразу создает внутреннее битовое поле нужного размера. Это позволяет избежать двойной работы (создания пустого поля и его последующего пересоздания).

Сложность: O(N), так как внутри вызывается конструктор TBitField.

Доступ к элементам (InsElem, DelElem, IsMember)

Алгоритм: Методы проверяют, что переданный элемент не выходит за пределы универсума (от 0 до MaxPower - 1). После проверки они делегируют работу внутреннему битовому полю (вызывают SetBit, ClrBit или GetBit).

Сложность: O(1), так как работа с битовым полем происходит за константное время.

Объединение, пересечение и дополнение (+, *, ~)

Алгоритм: благодаря тому, что множество хранится в виде битового вектора, теоретико-множественные операции сводятся к побитовым.

- Объединение +: вызывает побитовое ИЛИ для внутренних полей.
- Пересечение *: вызывает побитовое И.
- Дополнение ~: вызывает побитовое НЕ.

Особенность: для + и * результирующее множество создается с размером max(MaxPower1, MaxPower2), чтобы новое множество могло вместить все элементы обоих операндов.

Сложность: O(N), где N – количество машинных слов.

Объединение и разность с одиночным элементом (operator+ (int), operator- (int))

Алгоритм: создается полная копия текущего множества (вызывается конструктор копирования). Затем в эту копию добавляется (InsElem) или из нее удаляется (DelElem) нужный элемент. Исходное множество при этом не меняется (принцип иммутабельности для бинарных операторов).

Сложность: O(N), так как сначала нужно полностью скопировать битовое поле.

Ввод и вывод (<<, >>)

Алгоритм: Вывод перебирает все индексы от 0 до MaxPower, проверяет наличие через IsMember и печатает числа через запятую в фигурных скобках. Ввод считывает строку и посимвольно разбирает ее: пропускает скобки и запятые, накапливает число (путем value = value * 10 + (c - '0')) и добавляет его в множество.

Сложность: O(BitLen) при выводе и O(Length) при вводе (где Length – длина входной строки).

4. Результаты экспериментов и тестирования

Тестирование класса TBitField**:**

Конструкторы и деструктор: Проверено создание полей нулевой, нечетной (10, 20) и кратной 32 (64) длины. Проверена генерация исключения std::invalid_argument при попытке создать поле отрицательной длины.

Доступ к битам (SetBit**,** ClrBit**,** GetBit**):** Проверено, что установка бита не влияет на соседние. Проверены граничные случаи – работа с битами на стыке 32-битных слов (биты 0, 31, 32, 63), чтобы исключить ошибки при переходе между элементами массива pMem. Проверено, что при выходе за границы (например, SetBit(-1) или SetBit(100)) выбрасывается исключение std::out_of_range.

Управление памятью: Тесты копирования и присваивания подтвердили, что используется глубокое копирование (изменение копии не влияет на оригинал) и корректно обрабатывается самоприсваивание (a = a).

Побитовые операции: Операторы |, & и ~ проверены на корректность объединения, пересечения и инверсии. Отдельно проверено, что двойное отрицание ~(~a) возвращает исходное поле (что доказывает правильную работу обрезки «мусорных» битов в конце).

Сравнение: Проверка операторов == и != для полей с одинаковой и разной длиной.

Тестирование класса TSet**:**

Управление элементами: Проверены операции добавления (InsElem), удаления (DelElem) и проверки наличия (IsMember). Проверено выбрасывание исключений при попытке добавить элемент вне универсума.

Теоретико-множественные операции: Проверены объединение (+), пересечение (*) и дополнение (~). Убедились, что исходные множества при бинарных операциях не изменяются (иммутабельность).

Взаимодействие классов: Проверены конструкторы преобразования и операторы приведения типов (переход TSet <-> TBitField).

Все 23 теста пройдены успешно

Заключение

В ходе выполнения данной лабораторной работы были успешно решены поставленные задачи. Перечислим основные результаты:

- **Изучены принципы побитового хранения данных.** Реализован класс TBitField, позволяющий упаковывать 32 логических значения в одну переменную типа unsigned int. Это дает значительную экономию памяти по сравнению с использованием массива типа bool.
- **Освоены низкоуровневые операции.** На практике применены побитовые сдвиги (<<, >>), операции И (&), ИЛИ (|), НЕ (~) и маски для установки, сброса и проверки отдельных битов.
- **Освоен Google Test.** Написано 23 автоматических теста, покрывающих основной функционал и краевые случаи. Успешное прохождение всех тестов гарантирует надежность разработанной программы.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/